

Python Programming Language — Student Handbook

MODULE : FUNCTIONS*Topics 1 – 12***Instructor:** Duaa Fatima | **Course:** Python Programming Language**About This Module**

Is module mein aap Python ke Functions ko seekhenge — reusable code blocks jo kisi kaam ko baar baar bina dobara likhe run karte hain. Har topic short definition, quick Roman Urdu explanation, syntax, aur seedhe examples ke sath diya gaya hai.

In This Module

#	Topic
1	Defining Functions
2	Calling Functions
3	Parameters
4	Arguments
5	Return Statement
6	Default Parameters
7	Keyword Arguments
8	Variable-Length Arguments (*args)
9	Keyword Variable Arguments (**kwargs)
10	Lambda Functions
11	Recursion
12	Scope (Local & Global)
★	Module Summary, Revision Sheet, Viva Questions, Practice Questions, Coding & Debugging Exercises, Mini Assignment, Mini Project

1

Defining Functions

DEFINITION

A function definition is a reusable block of code, introduced by the def keyword, that performs a specific task when called.

Yeh Kya Hai

def keyword se hum apna khud ka function bana sakte hain jo dobara dobara use ho sake.

Kyun Use Hota Hai

Code repeat likhne ki bajaye ek jaga define kar ke bar bar reuse karne ke liye.

Kaam Kaise Karta Hai

def function_name(): likh kar neeche indented block mein code likha jata hai.

Syntax

PYTHON CODE

```
def function_name():  
    # code block  
    pass
```

Examples

Example 1

PYTHON CODE

```
def greet():  
    print("Hello!")  
  
greet()
```

OUTPUT

Hello!

Explanation: greet() function define hua, call hone par 'Hello!' print hua.

Example 2

PYTHON CODE

```
def add_numbers():  
    print(5 + 3)
```

```
add_numbers()
```

OUTPUT

```
8
```

Explanation: Function ke andar likha code sirf call hone par run hota hai.

Real-World Example

Website login ke baad welcome message dikhane wala function:

PYTHON CODE

```
def welcome_user():
    print("Welcome to MHK Academy Portal!")

welcome_user()
```

OUTPUT

```
Welcome to MHK Academy Portal!
```

Explanation: Function define karke portal ke har login par yehi welcome message reuse ho sakta hai.

IMPORTANT NOTES

- Function define karne se code run nahi hota, sirf call karne se hota hai.
- Function name variable naming rules follow karta hai.

COMMON MISTAKES

- Function call karna bhool jana.
- Indentation ghalat rakhna, jis se IndentationError aata hai.

KEY POINTS

- def keyword se function banta hai.
- Function reusable code block hota hai.

2

Calling Functions

DEFINITION

Calling a function means executing the code inside a previously defined function by writing its name followed by parentheses.

Yeh Kya Hai

Function call karna matlab uska naam likh kar () lagana, taake uske andar ka code chal sake.

Kyun Use Hota Hai

Bina call kiye function ka code kabhi execute nahi hota, is liye call karna zaroori hai.

Kaam Kaise Karta Hai

function_name() likhte hi Python us function ki definition dhoond kar uska body run karta hai.

Syntax

PYTHON CODE

```
function_name()
```

Examples**Example 1**

PYTHON CODE

```
def say_hi():  
    print("Hi there!")  
  
say_hi()  
say_hi()
```

OUTPUT

```
Hi there!  
Hi there!
```

Explanation: Function ko do dafa call kiya, is liye output do dafa print hua.

Example 2

PYTHON CODE

```
def show_date():  
    print("Today is a great day")
```

```
show_date()
```

OUTPUT

Today is a great day

Explanation: Function call hote hi uska code turant execute hota hai.

Real-World Example

Ek app mein 'send_notification' function ko har naye order par call karna:

PYTHON CODE

```
def send_notification():  
    print("Order received! Notification sent.")  
  
send_notification()
```

OUTPUT

Order received! Notification sent.

Explanation: Jab bhi order aaye, yehi function call ho kar notification bhej deta hai.

IMPORTANT NOTES

- Function use ke baad call ho sakta hai jab wo pehle define ho chuka ho.
- Ek function ko jitni martaba chahen call kar sakte hain.

COMMON MISTAKES

- Function ko define karne se pehle call karna (NameError aata hai).
- Parentheses () lagana bhool jana.

KEY POINTS

- Call karne se function ka code run hota hai.
- function_name() syntax se call hota hai.

3

Parameters

DEFINITION

Parameters are the named variables listed in a function's definition that receive values when the function is called.

Yeh Kya Hai

Parameters function definition mein diye gaye placeholders hain jo baad mein value receive karte hain.

Kyun Use Hota Hai

Function ko dynamic banane ke liye — taake wo alag alag values par kaam kar sake.

Kaam Kaise Karta Hai

Function name ke andar parentheses mein naam likh dete hain, jo call ke waqt value paate hain.

Syntax

PYTHON CODE

```
def function_name(parameter1, parameter2):  
    # use parameter1, parameter2
```

Examples**Example 1**

PYTHON CODE

```
def greet(name):  
    print("Hello", name)  
  
greet("Ali")
```

OUTPUT

Hello Ali

Explanation: 'name' parameter hai jisay 'Ali' value mili.

Example 2

PYTHON CODE

```
def add(a, b):  
    print(a + b)  
  
add(4, 6)
```

OUTPUT

10

Explanation: a aur b parameters ne 4 aur 6 values receive kin.

Real-World Example

Har customer ka naam le kar personalized bill message banana:

PYTHON CODE

```
def bill_message(customer_name):  
    print(customer_name, "your bill is ready.")  
  
bill_message("Sara")
```

OUTPUT

Sara your bill is ready.

Explanation: Parameter 'customer_name' se har customer ke liye alag message ban jata hai.

IMPORTANT NOTES

- Parameters function definition mein hote hain.
- Ek function mein multiple parameters comma se separate hote hain.

COMMON MISTAKES

- Parameters aur arguments ko ek jaisa samajhna.
- Zaroori parameter na dena (TypeError aata hai).

KEY POINTS

- Parameters function ko values receive karne dete hain.
- Function definition mein likhe jate hain.

4

Arguments

DEFINITION

Arguments are the actual values passed to a function's parameters when the function is called.

Yeh Kya Hai

Arguments wo actual values hain jo function call karte waqt parameters ko di jati hain.

Kyun Use Hota Hai

Function ko real data dene ke liye, taake wo us data par kaam kar sake.

Kaam Kaise Karta Hai

Call karte waqt parentheses ke andar jo values likhi jati hain wo order se parameters ko assign ho jati hain.

Syntax

PYTHON CODE

```
function_name(argument1, argument2)
```

Examples**Example 1**

PYTHON CODE

```
def multiply(a, b):  
    print(a * b)  
  
multiply(3, 4)
```

OUTPUT

```
12
```

Explanation: 3 aur 4 arguments hain jo a aur b parameters ko mile.

Example 2

PYTHON CODE

```
def show_info(name, age):  
    print(name, age)  
  
show_info("Ali", 20)
```

OUTPUT

Ali 20

Explanation: Arguments order ke mutabiq parameters ko assign hote hain.

Real-World Example

Ek calculator function ko user ke di gayi do numbers ke sath call karna:

PYTHON CODE

```
def add(a, b):  
    print(a + b)  
  
num1 = 15  
num2 = 25  
add(num1, num2)
```

OUTPUT

40

Explanation: Variables num1 aur num2 arguments ki tarah function ko pass hue.

IMPORTANT NOTES

- Arguments call statement mein diye jate hain.
- Arguments ki tadaad parameters ke barabar honi chahiye.

COMMON MISTAKES

- Kam ya zyada arguments dena (TypeError aata hai).
- Arguments ka order ghalat rakhna.

KEY POINTS

- Arguments actual values hain.
- Call karte waqt pass ki jati hain.

5

Return Statement

DEFINITION

The return statement ends a function's execution and sends a value back to the caller.

Yeh Kya Hai

return statement function se ek value wapas bhejta hai, taake wo aage use ho sake.

Kyun Use Hota Hai

Jab function ka result sirf print nahi, balke store ya aage use karna ho.

Kaam Kaise Karta Hai

return value likhne se function turant khatam ho jata hai aur wo value call wali jagah wapas mil jati hai.

Syntax

PYTHON CODE

```
def function_name():  
    return value
```

Examples**Example 1**

PYTHON CODE

```
def square(n):  
    return n * n  
  
result = square(5)  
print(result)
```

OUTPUT

25

Explanation: square() ne 25 return kiya jo result variable mein store hua.

Example 2

PYTHON CODE

```
def get_greeting():  
    return "Hello!"  
  
msg = get_greeting()
```

```
print(msg)
```

OUTPUT

Hello!

Explanation: return ki gayi value msg mein store hoke print hui.

Real-World Example

Ek function jo total price calculate kar ke wapas bheje, print nahi kare:

PYTHON CODE

```
def total_price(price, qty):  
    return price * qty  
  
amount = total_price(200, 3)  
print("Amount:", amount)
```

OUTPUT

Amount: 600

Explanation: return se mila result 'amount' mein store ho kar aage use hua.

IMPORTANT NOTES

- return ke baad function ka code execute nahi hota.
- Function bina return ke None return karta hai.

COMMON MISTAKES

- print() aur return ko ek jaisa samajhna.
- return ki value ko store karna bhool jana.

KEY POINTS

- return function se value wapas bhejta hai.
- return function ko turant khatam kar deta hai.

6

Default Parameters

DEFINITION

Default parameters are function parameters that assume a specified default value if no argument is provided for them during the call.

Yeh Kya Hai

Default parameter wo hai jise pehle se ek default value di gayi ho, agar call mein value na di jaye.

Kyun Use Hota Hai

Optional values dene ke liye, taake function bina us argument ke bhi kaam kar sake.

Kaam Kaise Karta Hai

Parameter = default_value likh kar default set hota hai, jo tab use hota hai jab argument na diya jaye.

Syntax

PYTHON CODE

```
def function_name(param=default_value):
    # code
```

Examples

Example 1

PYTHON CODE

```
def greet(name="Guest"):
    print("Hello", name)

greet()
greet("Ali")
```

OUTPUT

```
Hello Guest
Hello Ali
```

Explanation: Pehli call mein argument nahi diya to default 'Guest' use hua, dusri mein 'Ali' diya to wahi use hua.

Example 2

PYTHON CODE

```
def power(base, exp=2):
    print(base ** exp)
```

```
power(5)
power(2, 3)
```

OUTPUT

```
25
8
```

Explanation: Default exp=2 tab use hua jab exp nahi diya gaya.

Real-World Example

Ek discount function jahan default discount 5% ho agar user na batae:

PYTHON CODE

```
def apply_discount(price, discount=5):
    final = price - (price * discount / 100)
    print(final)

apply_discount(1000)
```

OUTPUT

```
950.0
```

Explanation: Default discount 5% use hua kyun ke koi custom value nahi di gayi.

IMPORTANT NOTES

- Default parameters function definition ke aakhir mein likhne chahiye.
- Call ke waqt di gayi value default ko override kar deti hai.

COMMON MISTAKES

- Non-default parameter ko default ke baad likhna (SyntaxError aata hai).
- Default value ko har call mein zaroori samajhna.

KEY POINTS

- Default parameter optional argument deta hai.
- = sign se default value set hoti hai.

7

Keyword Arguments

DEFINITION

Keyword arguments are arguments passed to a function preceded by the parameter name, allowing values to be matched by name rather than position.

Yeh Kya Hai

Keyword arguments mein hum parameter ka naam likh kar value dete hain, order matter nahi karta.

Kyun Use Hota Hai

Jab function mein bohot se parameters hon aur order yaad rakhna mushkil ho.

Kaam Kaise Karta Hai

Call karte waqt parameter_name=value likhte hain, jis se Python sahi parameter ko value assign kar deta hai.

Syntax

PYTHON CODE

```
function_name(param1=value1, param2=value2)
```

Examples**Example 1**

PYTHON CODE

```
def info(name, age):  
    print(name, age)  
  
info(age=20, name="Ali")
```

OUTPUT

```
Ali 20
```

Explanation: Order ulta likhne ke bawajood values sahi parameters ko mili, kyun ke naam se pass ki gayin.

Example 2

PYTHON CODE

```
def show(a, b, c):  
    print(a, b, c)  
  
show(c=3, a=1, b=2)
```

OUTPUT

1 2 3

Explanation: Keyword arguments se order ki zaroorat khatam ho jati hai.

Real-World Example

Ek registration function jahan clarity ke liye keyword arguments use kiye jayen:

PYTHON CODE

```
def register(name, city):
    print(name, "from", city)

register(city="Karachi", name="Sara")
```

OUTPUT

Sara from Karachi

Explanation: Keyword arguments se code zyada readable ban jata hai, order ki fikar nahi rehti.

IMPORTANT NOTES

- Keyword arguments se code zyada readable hota hai.
- Positional aur keyword arguments ek sath bhi use ho sakte hain.

COMMON MISTAKES

- Galat parameter name likhna (TypeError aata hai).
- Positional argument ko keyword argument ke baad likhna (SyntaxError aata hai).

KEY POINTS

- Keyword arguments naam se pass hote hain.
- Order ki zaroorat nahi rehti.

8

Variable-Length Arguments (*args)

DEFINITION

**args allows a function to accept an arbitrary number of positional arguments, collected into a tuple inside the function.*

Yeh Kya Hai

*args se function unlimited (koi bhi tadaad ke) positional values accept kar sakta hai.

Kyun Use Hota Hai

Jab pehle se pata na ho ke kitne arguments diye jayenge.

Kaam Kaise Karta Hai

*args sab extra positional values ko ek tuple mein jama kar deta hai, jise loop se access kiya ja sakta hai.

Syntax

PYTHON CODE

```
def function_name(*args):  
    # args is a tuple
```

Examples

Example 1

PYTHON CODE

```
def total(*args):  
    print(sum(args))  
  
total(1, 2, 3)  
total(5, 10)
```

OUTPUT

```
6  
15
```

Explanation: *args ne har call mein di gayi sari values ko tuple mein le liya aur sum() se jama kiya.

Example 2

PYTHON CODE

```
def show_all(*args):  
    for item in args:
```

```
print(item)

show_all("a", "b", "c")
```

OUTPUT

```
a
b
c
```

Explanation: Loop se args tuple ki har value print hui.

Real-World Example

Ek function jo kitne bhi products ke prices le kar total bill nikale:

PYTHON CODE

```
def total_bill(*prices):
    print("Total:", sum(prices))

total_bill(500, 250, 100)
```

OUTPUT

```
Total: 850
```

Explanation: *prices ne kitni bhi prices le kar unka total nikal diya, chahe products ki tadaad koi bhi ho.

IMPORTANT NOTES

- *args naam koi bhi ho sakta hai, sirf * zaroori hai (jese *nums).
- *args hamesha ek tuple hota hai.

COMMON MISTAKES

- *args ko dictionary samajhna (yeh tuple hota hai).
- *args ke baad normal positional parameter likhna galat order mein.

KEY POINTS

- *args unlimited positional arguments leta hai.
- Andar tuple ki tarah store hote hain.

9

Keyword Variable Arguments (**kwargs)

DEFINITION

***kwargs allows a function to accept an arbitrary number of keyword arguments, collected into a dictionary inside the function.*

Yeh Kya Hai

****kwargs** se function unlimited keyword arguments accept kar sakta hai, jo dictionary ki tarah store hote hain.

Kyun Use Hota Hai

Jab named values ki tadaad pehle se maloom na ho, jese form data ya settings.

Kaam Kaise Karta Hai

****kwargs** sab extra key=value pairs ko ek dictionary mein jama kar deta hai.

Syntax

PYTHON CODE

```
def function_name(**kwargs):  
    # kwargs is a dictionary
```

Examples

Example 1

PYTHON CODE

```
def show_info(**kwargs):  
    print(kwargs)  
  
show_info(name="Ali", age=20)
```

OUTPUT

```
{'name': 'Ali', 'age': 20}
```

Explanation: ****kwargs** ne dono keyword arguments ko dictionary mein jama kar liya.

Example 2

PYTHON CODE

```
def show_info(**kwargs):  
    for key, value in kwargs.items():  
        print(key, ":", value)
```

```
show_info(city="Lahore", zip=54000)
```

OUTPUT

```
city : Lahore
zip : 54000
```

Explanation: .items() se kwargs dictionary ki har key-value pair print hui.

Real-World Example

Ek user profile function jo kitni bhi extra details (city, phone, etc.) accept kar sake:

PYTHON CODE

```
def create_profile(**details):
    print(details)

create_profile(name="Sara", city="Karachi", phone="0300")
```

OUTPUT

```
{'name': 'Sara', 'city': 'Karachi', 'phone': '0300'}
```

Explanation: **details har naya field bina function ko dobara likhe accept kar leta hai.

IMPORTANT NOTES

- **kwargs hamesha dictionary hota hai.
- **kwargs ko *args ke baad hi likha ja sakta hai.

COMMON MISTAKES

- **kwargs ko tuple samajhna (yeh dictionary hota hai).
- *args aur **kwargs ka order ghalat rakhna (args pehle, kwargs baad mein).

KEY POINTS

- **kwargs unlimited keyword arguments leta hai.
- Andar dictionary ki tarah store hote hain.

10

Lambda Functions

DEFINITION

A lambda function is a small, anonymous function defined using the lambda keyword, capable of taking any number of arguments but only one expression.

Yeh Kya Hai

Lambda function ek chota, bina naam wala (anonymous) function hai jo ek line mein likha jata hai.

Kyun Use Hota Hai

Jab ek simple, short function chahiye ho jo sirf ek dafa ya thori der ke liye use ho.

Kaam Kaise Karta Hai

lambda arguments: expression likha jata hai, jo expression ka result return kar deta hai.

Syntax

PYTHON CODE

```
function_name = lambda arguments: expression
```

Examples**Example 1**

PYTHON CODE

```
square = lambda x: x * x  
print(square(5))
```

OUTPUT

25

Explanation: lambda ne $x * x$ expression evaluate karke 25 return kiya.

Example 2

PYTHON CODE

```
add = lambda a, b: a + b  
print(add(3, 7))
```

OUTPUT

10

Explanation: Lambda ne do arguments le kar unka sum return kiya, def function jaisa hi kaam kiya lekin ek line mein.

Real-World Example

Products ki list ko price ke hisab se sort karna (lambda key ke tor par):

PYTHON CODE

```
products = [("Pen", 50), ("Book", 200), ("Bag", 800)]
products.sort(key=lambda item: item[1])
print(products)
```

OUTPUT

```
[('Pen', 50), ('Book', 200), ('Bag', 800)]
```

Explanation: lambda ne har item ki price (index 1) ke hisab se sorting karwa di.

IMPORTANT NOTES

- Lambda mein sirf ek expression likha ja sakta hai, statements nahi.
- Lambda aksar sort(), filter(), map() ke sath use hota hai.

COMMON MISTAKES

- Lambda mein multiple statements likhne ki koshish karna (SyntaxError aata hai).
- Complex logic ke liye lambda use karna, jabke normal function behtar hota.

KEY POINTS

- Lambda anonymous, one-line function hai.
- Short aur simple operations ke liye best hai.

11**Recursion****DEFINITION**

Recursion is a technique in which a function calls itself in order to solve a problem by breaking it into smaller subproblems.

Yeh Kya Hai

Recursion mein ek function apne aap ko call karta hai taake bara problem chote hisso mein solve ho.

Kyun Use Hota Hai

Jab problem repeatedly khud jaisi chote problem mein toota ja sake, jese factorial ya fibonacci.

Kaam Kaise Karta Hai

Function har call mein chota input ke sath khud ko call karta hai, jab tak base case (rukne ki condition) na aa jaye.

Syntax**PYTHON CODE**

```
def function_name(n):  
    if base_case:  
        return value  
    return function_name(smaller_n)
```

Examples**Example 1****PYTHON CODE**

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5))
```

OUTPUT

120

Explanation: factorial() ne khud ko chote n ke sath call kiya jab tak n==1 (base case) nahi aaya, phir sab multiply hue: $5*4*3*2*1=120$.

Example 2

PYTHON CODE

```
def countdown(n):
    if n == 0:
        print("Done!")
        return
    print(n)
    countdown(n - 1)

countdown(3)
```

OUTPUT

```
3
2
1
Done!
```

Explanation: Har call n ko 1 kam karke khud ko dobara call karta hai jab tak $n=0$ na ho jaye.

Real-World Example

Kisi folder ke andar sab sub-folders ginne jese systems recursion use karte hain — yahan sum of digits ka example:

PYTHON CODE

```
def sum_digits(n):
    if n == 0:
        return 0
    return n % 10 + sum_digits(n // 10)

print(sum_digits(123))
```

OUTPUT

```
6
```

Explanation: Function har dafa last digit nikal kar baqi number ke sath khud ko call karta hai, jab tak number khatam na ho.

IMPORTANT NOTES

- Har recursive function mein ek base case zaroori hai, warna infinite recursion ho jati hai.
- Recursion elegant hoti hai lekin loops se zyada memory use karti hai.

COMMON MISTAKES

- Base case likhna bhool jana (RecursionError aata hai).
- Har call mein input ko chota na karna, jis se function kabhi na rukay.

KEY POINTS

- Recursion function ka khud ko call karna hai.
- Base case hamesha zaroori hota hai.

12

Scope (Local & Global)

DEFINITION

Scope determines the region of a program where a variable is accessible; local scope is limited to inside a function, while global scope applies throughout the program.

Yeh Kya Hai

Scope batata hai ke variable kahan accessible hai — sirf function ke andar (local) ya poore program mein (global).

Kyun Use Hota Hai

Variables ko sahi jaga par manage karne ke liye, taake naming conflicts na hon.

Kaam Kaise Karta Hai

Function ke andar bana variable local hota hai (bahar accessible nahi), jabke function se bahar bana variable global hota hai (har jaga accessible).

Syntax

PYTHON CODE

```
def function_name():  
    local_var = value    # sirf andar accessible  
global_var = value    # har jaga accessible
```

Examples

Example 1

PYTHON CODE

```
def show():  
    x = 10    # local  
    print(x)  
  
show()
```

OUTPUT

10

Explanation: x sirf function ke andar accessible hai, bahar se access nahi ho sakta.

Example 2

PYTHON CODE

```
y = 100    # global
```

```
def show():
    print(y)

show()
```

OUTPUT

```
100
```

Explanation: y global hai is liye function ke andar bhi accessible hai.

Real-World Example

Ek app mein global counter jo har function call par update ho:

```
PYTHON CODE
visit_count = 0

def new_visit():
    global visit_count
    visit_count += 1

new_visit()
new_visit()
print(visit_count)
```

OUTPUT

```
2
```

Explanation: 'global' keyword se function ke andar se bhi global variable ko update kiya ja saka.

IMPORTANT NOTES

- Function ke andar global variable modify karne ke liye 'global' keyword zaroori hai.
- Local variable function khatam hote hi memory se hat jata hai.

COMMON MISTAKES

- Local variable ko function ke bahar access karne ki koshish karna (NameError aata hai).
- 'global' keyword use kiye baghair function ke andar global variable update karna.

KEY POINTS

- Local scope: sirf function ke andar.
- Global scope: poore program mein accessible.

Module Summary

Is module mein humne Python Functions seekhe — def se function define karna, call karna, parameters aur arguments dena, aur return statement se value wapas lena. Default parameters aur keyword arguments ne function calls ko flexible aur readable banaya.

*args aur **kwargs se humne unlimited positional aur keyword arguments accept karna seekha. Lambda functions ne short one-line functions likhna sikhaya, recursion ne function ko khud ko call karna sikhaya, aur scope ne bataya ke variable kahan accessible hota hai (local ya global).

Quick Revision Sheet

Topic	Quick Syntax / Meaning
Defining Functions	def function_name(): ...
Calling Functions	function_name()
Parameters	def func(param1, param2):
Arguments	func(value1, value2)
Return Statement	return value
Default Parameters	def func(param=default):
Keyword Arguments	func(param=value)
*args	def func(*args): # tuple
kwargs	def func(kwargs): # dict
Lambda Functions	f = lambda x: x * x
Recursion	function calls itself + base case
Scope	local = inside function global = whole program

Viva / Interview Questions

1. Function kya hoti hai aur kyun use hoti hai?
2. Parameter aur argument mein kya farq hai?
3. return statement print() se kaise different hai?
4. Default parameter kya hota hai?
5. Keyword arguments ka faida kya hai?
6. *args aur **kwargs mein kya farq hai?
7. Lambda function kya hoti hai?
8. Recursion mein base case zaroori kyun hai?
9. Local aur global variable mein farq batayen.
10. Function bina return ke kya deta hai?

Mixed Practice Questions

1. `def add(a, b): return a + b` — `print(add(4, 5))` ka output kya hoga?
2. `def greet(name="Guest"): print(name)` — `greet()` ka output kya hoga?
3. `*args` kis data type mein values store karta hai?
4. `**kwargs` kis data type mein values store karta hai?
5. Lambda function mein kitne expressions likhe ja sakte hain?
6. Recursion mein agar base case na ho to kya hoga?
7. Function ke andar bana variable kis scope ka hota hai?
8. `def func(): return` — is function ka return value kya hoga?
9. `func(name="Ali", age=20)` — yeh kis type ka argument hai?
10. Global variable ko function ke andar update karne ke liye kaun sa keyword chahiye?

Coding Exercises

1. Ek function banayein jo do numbers le kar unka average return kare.
2. Ek function banayein jisme default parameter ho aur bina argument diye test karein.
3. `*args` use kar ke ek function banayein jo kitni bhi numbers ka maximum nikale.
4. `**kwargs` use kar ke ek function banayein jo student ki details print kare.
5. Ek lambda function banayein jo kisi number ko cube (power 3) kare.
6. Recursion use kar ke ek function banayein jo kisi number tak ke sab numbers ka sum nikale.
7. Ek global counter variable banayein aur usay ek function ke zariye increment karein.

Debugging Exercises

Neeche har code snippet mein ek bug (ghalti) chupi hui hai. Bug identify karein aur usay fix karein.

Debug Exercise 1

```
PYTHON CODE
def greet():
    print("Hello")

greet()
```

Debug Exercise 2

```
PYTHON CODE
def add(a, b):
    return a + b

print(add(5))
```

Debug Exercise 3

```
PYTHON CODE
def square(n)
    return n * n

print(square(4))
```

Debug Exercise 4

```
PYTHON CODE
def total(*args):
    print(args + 1)

total(1, 2, 3)
```

Debug Exercise 5

```
PYTHON CODE
def show():
    print(x)
    x = 10

show()
```

Mini Assignment

Title: Student Grade Calculator

Ek Python program likhein jo neeche di gayi requirements पूरी kare:

1. Ek function banayein jo student ke 3 subjects ke marks parameters ke tor par le.
2. Function total aur average calculate kar ke return kare.
3. Default parameter use karein jisme agar koi marks na diye jayen to unki value 0 ho.
4. Ek lambda function banayein jo average ke base par grade (A/B/C) decide kare.
5. Final result (total, average, grade) print karein.

Mini Project

Title: Simple Function-Based Calculator

Is mini project mein aapko ek calculator banana hai jo is module mein seekhi gayi function concepts ko practically use kare.

Requirements

1. Alag alag operations (add, subtract, multiply, divide) ke liye alag functions banayein.
2. *args use kar ke ek function banayein jo kitni bhi numbers ka sum nikale.
3. Default parameter use karein taake divide function mein divisor default 1 ho.
4. Ek lambda function banayein jo do numbers ka average nikale.
5. Recursion use kar ke ek function banayein jo kisi number ka factorial nikale.
6. Ek global 'history' list banayein aur function ke andar 'global' keyword se usay update karein.

Expected Learning Outcome

Is project ko complete karne ke baad student functions, parameters, arguments, *args, **kwargs, lambda, recursion, aur scope ko real calculator jaisi application mein confidently use kar sakega.